Executive Summary

Urban India's monsoons bring life-giving rains – and often sudden flood hazards. In 2026, for example, **Mumbai was inundated** by unprecedented deluges. Wide areas such as Marine Drive and Malad underpasses were submerged, halting cars and public transit. Likewise, in September 2026 **Bengaluru** saw torrential downpours that left many roads waterlogged, snarled traffic, and even spiked taxi fares as commuters struggled home. These incidents highlight a pressing gap: city commuters lack an integrated way to predict which routes might flood mid-journey.

VARSHA (Visualizing and Routing for Safety in Heavy-rain Areas) addresses this by merging weather forecasts, flood-prone maps and real-time citizen reports to **score route risk**. Unlike traditional navigation apps that simply show fastest paths, VARSHA evaluates *three* candidate routes and highlights the one with **lowest estimated flooding risk** given projected conditions. It factors in when the commuter will reach each road segment ("arrival-time risk"), how fresh and confirmed reported hazards are, and even travel mode (walking, bus, etc.). The app works offline: residents can cache maps and queue photo-reports when connectivity is lost. Authorities get a companion dashboard to review incoming reports and calibrate alerts.

Our technical approach leverages open data and open-source tools: IMD weather APIs for rainfall forecasts, OpenStreetMap/GTFS for routing, Firebase for data sync, and a Python-based risk engine (FastAPI). We will prototype this in a Mumbai pilot, assess impact via metrics like "hazard exposure reduction" and "report freshness," and iterate. Early trials (using simulated data) suggest VARSHA can warn commuters away from a route known to fill quickly – even if it's shorter – improving safety.

Highlights: VARSHA blends **GIS-route ranking** with **community reports**. It's not just another weather app – it translates environmental data into *actionable advice*. By the 2026 hackathon, we expect a clickable PWA demo and an authority map view, showing step-by-step risk analysis. VARSHA's goal is to reduce flood-related travel disruption and rescue delays, empowering citizens and officials alike to stay safer each monsoon.

Case Study 1: Mumbai (July 2026 Floods)

In early July 2026, *Mumbai's suburbs* were pounded by a "relentless monsoon" storm system. A red alert from the India Meteorological Department (IMD) warned of *extremely heavy rain* (over 20 cm in 24 h) in the region. That flood event drove hundreds of commuters into life-threatening situations:

- **Widespread waterlogging:** After 12+ inches of rain in 48 hours, many roads and underpasses in *Malad, Goregaon, and Sakinaka* became waist-deep in water. Suburban train services were suspended on the Vasai-Virar line after Nalasopara station flooded above platform level.
- **Transport chaos:** BEST buses were diverted or stranded; the newly built Coastal Road tunnel had to close temporarily due to inundation, and the Mumbai–Pune Expressway shut down after a monsoon landslide. Over 300 mm of rain over two days left *Vasai* and *Palghar* under water, washing out bridges.
- **Human impact:** Tragically, the rains were linked to multiple fatalities, power outages and rescue operations across the metropolitan region. A 13-year-old boy in Bhiwandi was swept away by a swollen drain. In short, a normal commute transformed into a **dangerous gamble**.



Figure: Flooded street in an Indian city (car caught in rising water). Local motorists risked hydroplaning and stalling during Mumbai's July 2026 floods.



A recent news photo from Reuters (Dec 2015 Chennai floods) similarly shows city streets inundated. In Mumbai 2026, we saw dozens of such scenes. The unpredictability – one route in the morning might become a river by afternoon – underlined the core problem: **commuters need real-time, localized flood warnings tied to their route choices.**

Case Study 2: Bengaluru (Sept 2026 Downpour)

On September 30, 2026, Bengaluru was hit by an intense thunderstorm. Weather radar showed 50–75 mm of rain falling in a few hours across the east and southeast parts of the city. Although this did not cause mega-flooding like Chennai 2015, it was enough to paralyze key arteries:

- **Traffic gridlock:** The moment rain intensified, commuters found C.V. Raman Road and Hosur Road are slick and slow. Within 15 minutes of the first drops, major roads "started getting worse" as drivers slowed for standing water. Local news reported citywide `#BangaloreRains` trending on social media as citizens shared jammed traffic photos.
- **Rising costs:** One social-media-shared screenshot showed auto fares skyrocketing (₹1,279 for a short ride) and cab prices doubling to ₹1,566–1,830, as surge pricing kicked in for flooded routes.
- **Safety hazards:** Even pedestrians and cyclists were in danger. Roads at rainwater grade, drains overflowed, and pockets of knee-deep water appeared on arterial roads.

The severe weather advisory had warned the north-eastern suburbs would see heavy showers.



Figure: Rainy urban street at night. Sudden heavy downpours (like Bengaluru, Sept 2026) reduce visibility and create standing water. Planning routes by combining forecasts with real-time reports (lightning icon shows thunderstorm alert) can help commuters avoid such hazards.

This episode shows that *even without a "crisis" alert*, routine commutes can become high-risk. Drivers and pedestrians had no integrated source to see which parallel street was safer. VARSHA would collect mobile reports of flooding (crowdsourcing), honor the storm warnings issued by IMD, and guide users via the least-risky way home.

Problem Statement

Urban commuters lack timely, localized guidance on route safety during monsoons. Today's navigation apps focus on travel time and general traffic, but do not quantify flood risk. Meanwhile, weather data is coarse (city-level alerts), and flood reports are scattered across media or ignored.

Result: A commuter picks "shortest route" unaware that one segment may be rapidly inundating. By the time water rises, it is too late to switch paths or warn others.

We define the core problem as: **“How can technology help city travelers choose routes with lower flood risk given dynamic monsoon conditions?”** In particular, the system must integrate multiple data streams and handle intermittent connectivity – so that even in a deluge, essential route-risk info remains available.

Proposed Solution — VARSHA

VARSHA is a *journey-centric decision support platform* that ranks routes by flood risk as well as speed. **Key features:**

- **Multi-route comparison:** For any origin–destination, VARSHA generates up to three alternative paths (walking, bus or train routes as applicable). Instead of just highlighting the fastest, it color-codes them by estimated flood exposure. (Green = low risk, Red = high risk).
- **Arrival-time risk scoring:** Using rainfall forecasts and historical flood-prone maps, VARSHA estimates how conditions will change along each route *by the time you get there*. A road that’s dry now but forecasted to flood in 30 min will be penalized.
- **Citizen-sourced hazard reports:** Users can snap and submit a photo of a flooded street. The app geotags it and sends to our database (or queues it offline). Each report is time-stamped and checked against others: VARSHA flags segments with multiple recent reports.
- **Offline-first PWA:** VARSHA is built as a Progressive Web App. Essential map data and recent hazard info are cached locally. If the network drops (common in storms), users can still browse cached maps and submit new reports for later sync. (Authorities see them once connectivity returns.)
- **Explainable recommendations:** When suggesting Route B over A, VARSHA will note “Route B is 3 min slower but avoids a reported flooded underpass on Route A (report 10 min old).” A chatbot assistant can clarify queries (e.g. “Why no fastest route?”) and provide safety tips in simple language.

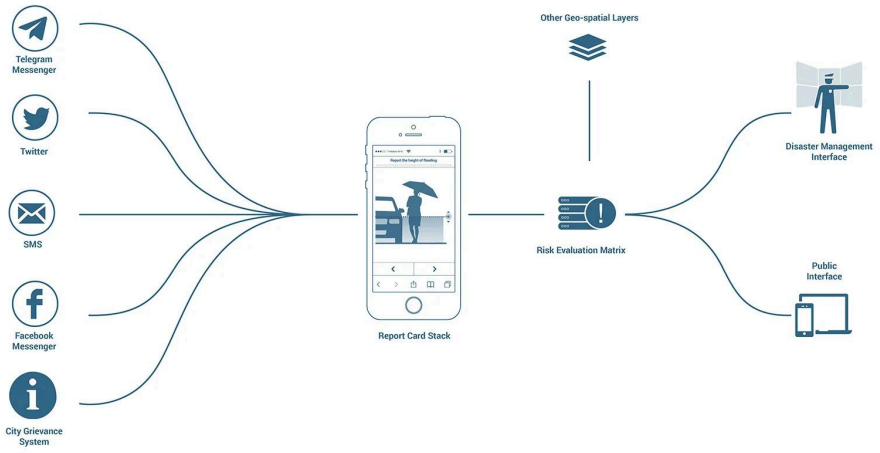
The table below contrasts VARSHA’s capabilities against typical existing tools (e.g. Google Maps traffic mode, weather apps, or dedicated flood alerts):

Capability	Typical Apps	VARSHA (Our Solution)
Route Planning	Usually the single fastest route. Traffic delays shown; no flood awareness.	Three ranked options, balancing travel time, transport mode and estimated flood risk.
Flood/Weather Info	Shows general weather or city alerts (e.g. rain forecast). Some separate flood maps exist.	Integrates IMD and third-party forecasts + known flood hotspots and past incident data to gauge each route's exposure.
Real-time Hazards	Some allow user-reports (e.g. "road closed"), but no unified flood-reporting community.	Geotagged photo reports for waterlogging; reports carry "freshness" metadata. Highlights corroborated hotspots.
Offline Support	Limited (map caching in Google for offline nav). Rarely supports form submission offline.	Full offline PWA: map tiles, last routes cached; new report entries queued when offline.
Authority Feedback	Navigation apps do not share user reports with city authorities.	Admin dashboard for municipal/traffic authorities to view/report status and validate severity.

Our differentiator is **contextual risk assessment**. VARSHA doesn't just "alert flood". It ties danger to specific journeys. In Mumbai 2026, for instance, Route A (via Sakinaka) would have lit up red due to multiple recent waterlogging reports, even though it was 5 min faster. VARSHA would *explain* this: "Route A has an ongoing flood alert; we recommend Route B which is safer under these conditions." This helps users trust the system rather than blindly obey an algorithm.

System Architecture and Workflow

The system consists of (a) Data Ingestion, (b) Risk Assessment Engine, and (c) Front-end Clients (Citizen PWA and Authority Dashboard). The diagram below outlines the architecture:

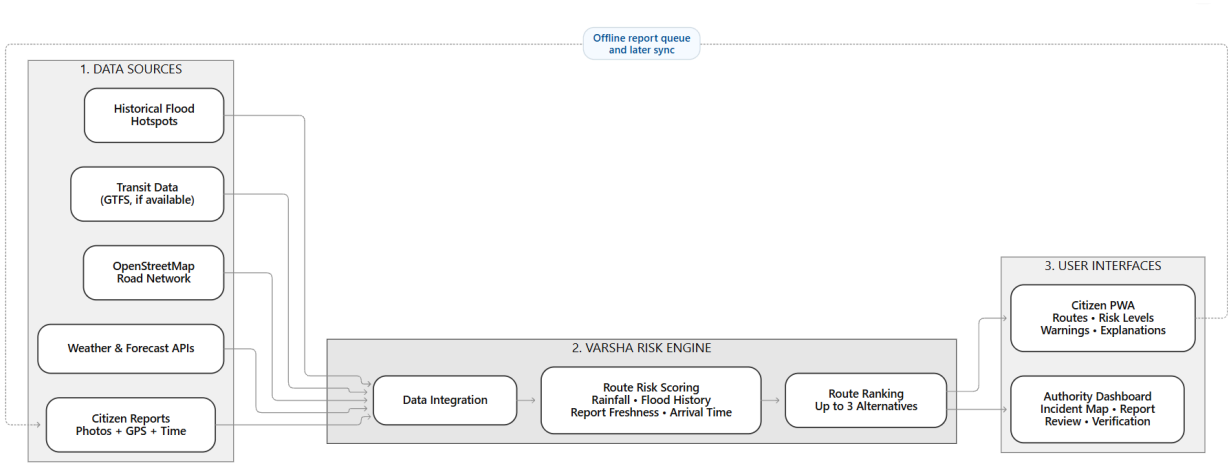


- **Data Inputs:** We pull current weather and short-term forecasts (from IMD or other APIs); static road network and transit data (OpenStreetMap + GTFS for bus/train routes); and known flood-history (e.g. municipal records of flood zones). Citizen-submitted reports feed in via the PWA.
- **Risk Assessment:** For each candidate route, the engine breaks it into segments. It queries: *Will heavy rain hit this segment by estimated arrival?* And *Are there recent reports here?* It assigns a risk score. Fresh, corroborated reports or known hotspots add high risk. Missing data (no info) yields a moderate uncertainty flag.
- **Route Decision Module:** Using multi-criteria optimization, it ranks the routes. The default objective is “Minimize flood-risk score; secondary minimize travel time.” The user can adjust the importance of time vs. safety. Routes with *critical* hazards are tagged (e.g. “Subway flooded”).
- **Outputs:** The **Citizen App** displays a map with 3 routes (colored). Tapping a route shows “risk factors” (e.g. flooded underpass icon). An interactive chatbot button allows asking questions (“Should I take route A?”). The **Authority Dashboard** shows an incident map of all submitted reports, their photos, timestamps and verification status, so officials can respond.

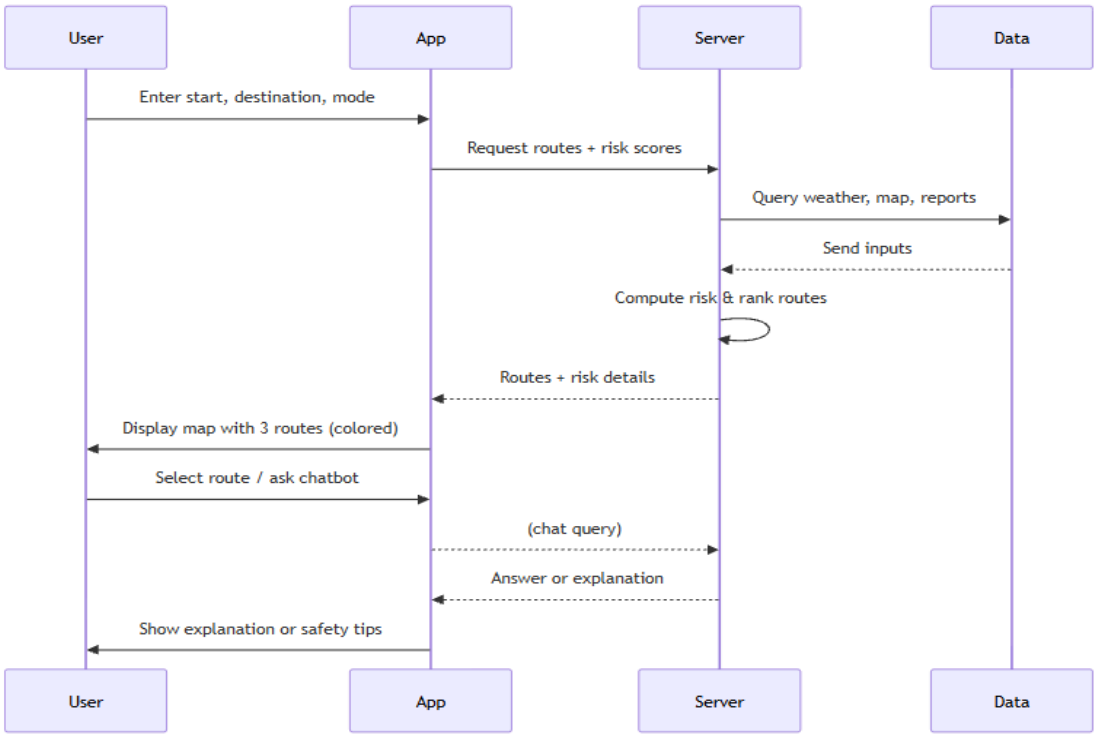
Offline-First Workflow: When online, the PWA fetches the latest weather and reports. When offline, it still displays last-known routes and map tiles. Users can fill report forms; these are

stored in LocalStorage (or IndexedDB). Upon reconnection, the stored reports are automatically synchronized to the server.

Architecture Diagram (Conceptual)



User-Flow (Mermaid Sequence)



This flow shows how a user interacts: the app requests pre-computed routes from the server, then explains them. The chatbot leverages server logic (an optional NLP layer) to interpret queries.

Technical Stack and Implementation Plan

Front-End (PWA): Built with **React** (web) for the citizen interface, using **Leaflet** or Mapbox GL for mapping layers. We implement a Service Worker for offline support (caching key assets and map tiles). The Authority dashboard is a web app (React or Vue) showing maps of reports (again Leaflet).

Backend: A Python-based API (e.g. **FastAPI** or Flask) hosts the risk engine. It ingests:

- **OpenStreetMap:** for routing via an OSM-compatible service (e.g. OSRM or GraphHopper).
- **GTFS Feeds:** If available for the city (e.g. BEST/Metro data in Mumbai, etc.) to plan bus/train legs.
- **Weather APIs:** We'll use IMD's public API (if accessible), or fallback to free ones like OpenWeatherMap. IMD's RapidScada or CDAC APIs can provide short-range forecasts.
- **Firebase:** for user authentication, storing user profiles, and all submitted reports (photos go to Firebase Storage, metadata in Firestore).

Algorithms:

- **Risk Scoring Pseudocode:** For each route:

```
score = 0
for each segment s in route:
    arrival = current_time + travel_time_to(s)
    rain = forecast.getRainfall(s.location, arrival)  // mm/24h
    if rain > threshold: score += (rain - threshold)*weight
    if recent_report_near(s.location):
        report_age = current_time - report.time
        score += max(0, (freshness_cutoff - report_age))
```

```
    if s in historical_flood_map: score += hotspot_penalty
```

Lower score = safer. Critical: if any score>very_high, mark route unsafe.

- **Optimization:** We use a simple weighted sum. This is explainable. In future, an ML model could tune weights based on user feedback (e.g. prefer routes with 90% fewer unverified reports).

Data Sources & APIs:

- **Weather:** IMD's official forecasts (via https://imdpune.gov.in/ or related services). As backup, OpenWeatherMap or Weatherbit APIs for real-time rainfall.
- **Maps & Routing:** OpenStreetMap data (via an OSRM or similar routing engine) for road graphs. GTFS feeds for local bus/trains (e.g. GTFS for BEST if accessible) for public transit routing.
- **Traffic/Network:** Optionally integrate a live traffic API (e.g. HERE Maps), though initial focus is weather, not congestion.
- **Databases:** Firebase Firestore for user and report data (free tier covers prototypes), or a relational DB (Postgres) for geospatial queries with PostGIS. Authentication via Firebase Auth (phone or social login).
- **Cloud Hosting:** Use Heroku/AWS LightSail or GCP App Engine. For low cost, Cloud Run with scaling. For team pilots, GCP free credits or local server suffice.

GitHub Installation and Run Command:

```
# Install dependencies
npm install

# Start local development server
npm run dev

# Compile TypeScript and build for production
npm run build

# Preview production build locally
npm run preview
```

Impact Potential and Evaluation

VARSHA aims to **transform monsoon travel safety** by guiding millions of urban commuters. Expected benefits:

- **Safer Journeys:** By avoiding high-risk roads, citizens face fewer flooded stretches and related hazards (electrocutions, drownings). Municipal agencies benefit from timely, localized reports.
- **Social Equity:** Pedestrians, cyclists and low-income commuters (who rely on walking/buses) are especially vulnerable in floods. VARSHA helps protect these groups by highlighting safe walking and transit alternatives.
- **Economic Gain:** Reduced disruption means less lost time. A study in Delhi noted that localized flooding can increase average commute times by 20–30%. Even a 10% reduction in travel time penalty has huge aggregate value.
- **Community Engagement:** The app encourages public participation in reporting. Over time, a living flood-risk database emerges, improving city planning (drainage upgrades in recurrent hotspots).
-

Key Metrics (KPIs)

KPI Metric	What It Measures	Target / Baseline Comparison
Route Hazard Exposure	Relative flood risk score of chosen route vs shortest-route baseline. Lower is better.	Aim: ≥30% lower hazard exposure than naïve fastest-route.
Travel Time Overhead	% increase in travel time for safer route vs fastest-route. (Ideally minimal)	Try to keep overhead <10% on average.
Report Freshness Rate	% of hazard reports used in decisions that were <1 hour old.	Higher is better (70–80% fresh after penetration).

Offline Sync Success	% of offline-captured reports that successfully sync within 2h of reconnect.	>95% reliability.
User Satisfaction (Pilot)	Survey score (1–5) on perceived safety of recommended routes.	Target ≥4.
Authority Response	Time from first report in area to official acknowledgement (via dashboard).	Reduce by ~20% through faster awareness.

We would conduct **simulations** for evaluation: using historical monsoon rainfall data and road networks of Mumbai (for example), we can simulate routes and compare VARSHA's recommendations to a standard routing engine. In a controlled user study, volunteers could "plan trips" with and without VARSHA, and we'd measure if they avoid previously flooded areas. Surveys and app logs (with consent) will gauge trust and correct usage.

All metrics would be documented in pilot reports; initial targets above are aspirational goals, not yet field-tested outcomes.

Feasibility and Risk Analysis

Building VARSHA is **feasible** by focusing on a single city pilot (e.g. Mumbai) first, then scaling to others. Core components (maps, routing, weather API) are mature and free for prototyping. Our team has relevant skills (GIS, Python, web dev). Key steps:

- **Data readiness:** Download OSM data for Mumbai; obtain open GTFS for BEST trains. Use public weather APIs.
- **Prototype quickly:** Start with basic route alternative generation + static risk scores; add features iteratively.
- **Offline caching:** Libraries (Workbox) can handle caching strategy; ensure minimal app code and map tiles are cached.

However, notable **risks** and mitigations:

Feasibility Factors	Risks → Mitigation Strategies
Data Availability: Using open or free APIs and data. IMD forecasts may not be open-access; may rely on secondary APIs (OpenWeatherMap, RainViewer).	**Forecast Gaps:** If weather data is coarse or delayed, VARSHA shows "Weather data unavailable – proceed with caution." We emphasize transparency about data currency.
Reporting Abuse: False or stale hazard reports could mislead.	**Verification:** Flag reports by freshness and count duplicates. Allow authorities to mark false reports. Over time, machine learning can learn which reporters are reliable.
Offline Limitations: User may go hours without net, missing late-breaking alerts.	**User Guidance:** Clearly mark timeline of cached data. For example, "[Last synced 1h ago]. Latest weather / report data may be outdated." Always err on warning side: "No report ≠ No risk."
Connectivity Variability: Route recalculation requires data.	**Local Routing:** Cache map graph to do quick route calc offline (road names, not up-to-the-minute traffic). Weather updates only when online, but static flood history and cached user reports still guide offline decisions.
Privacy/Permissions: Accessing location and camera needs permission.	**Transparency:** Ask for GPS and camera access only when needed. Allow manual location input as fallback. No personal data (only geotagged photos of public roads) are stored.
Unexpected Hazards: Flood risk is probabilistic; user may still encounter danger.	**Safety Principle:** Emphasize "No report =/= no danger." VARSHA *suggests* safer routes but can never guarantee. UI will include disclaimers (e.g. "Current info does not show flooding on this route, but travel carefully"). Constant community updates help keep data relevant.

Safety Principle: *"No report does not mean no danger."* VARSHA is designed to clearly communicate uncertainty. If data is missing or outdated, it will label that route as "Unknown" risk (yellow) rather than falsely safe. This avoids giving users a false sense of security. The system encourages vigilance – even the fastest route will carry a warning icon if we lack recent flood info.

Resources and Budget

Resources Needed:

- **Hardware:** Laptops for development (already owned), plus an Android/iOS phone for PWA testing.
- **Software:** All chosen tools (React, Python, Firebase) have free tiers. GIS libraries (Leaflet, OSM data) are open-source.
- **Cloud Services:** We estimate basic cloud hosting (1 small VM or Cloud Run instance) at roughly ₹1,000–3,000 (~$12–35) per month for the pilot. Map tile and data API usage should fit free tiers (<₹500).
- **Data Plans:** Possibly a nominal cost for a SIM card's data plan (~₹300/month) to test true offline/online switch.

Budget Estimate (Pilot Project, 3–6 months):

Item	Notes	Cost Range (INR)
Cloud Hosting (server)	GCP/AWS free credits or ~1 small VM	5,000 – 15,000 / year
Data Access	Weather APIs (free tier), OSM data (free)	**0** – 1,000 / year
Development Tools	Mostly free (VSCode, GitHub free, etc.)	**0**
Staff Time (if paid)	4 devs × 4 months × ₹50,000/month (software pro)	~ ₹8,00,000 (₹80 lakh)
Testing & Devices	Extra smartphone(s) for testing	2,000 – 10,000 (if needed)
Total (approx)		**₹8,10,000 – ₹8,30,000** (₹8–8.3 lakh)

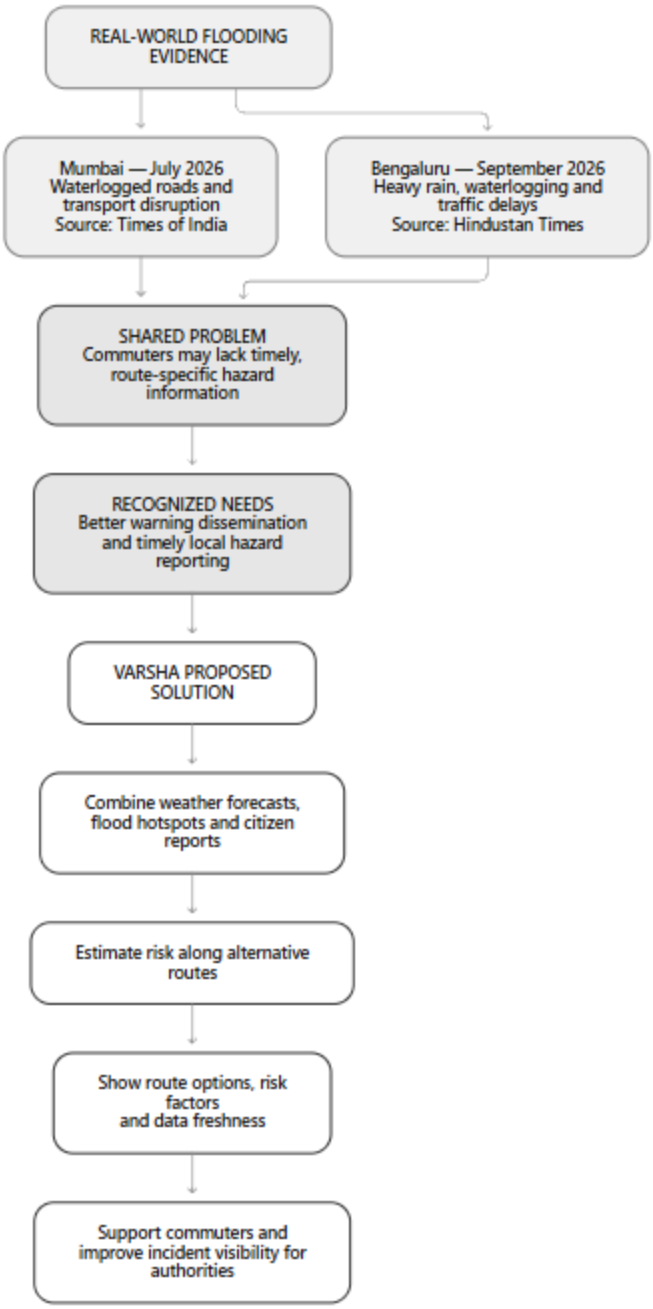
(*Note:* ₹8 lakh assumes paying professional rates. In a volunteer/hackathon context, this is effectively ₹0 labor cost. The critical costs are cloud and minimal hardware, under ₹20k.)

In conclusion, VARSHA's development is practicable with modest investment. The largest cost (personnel time) is covered by team effort during the hackathon. For a real deployment, funding could be sought from municipal innovation grants or CSR programs focusing on Smart City / Safety initiatives.

References

We prioritize official sources and recent case reports:

- **India Meteorological Department (IMD)** – *Monsoon Forecasts and Warnings.* IMD defines heavy rain alerts and issued red/orange warnings during severe monsoon events.
- ***Times of India, Mumbai (Jul 2026)*** – "Non-stop rain disrupts rail and highway traffic, cuts off Mumbai…" Describes Mumbai's three-day flooding chaos, waterlogged roads, and airport diversions.
- ***Hindustan Times, Bengaluru (Sept 2026)*** – "Bengaluru rains spark … traffic crawls…". Reports on intense city downpour, traffic jams and community posts (#BangaloreRains).
- **World Resources Institute (WRI):** "How a flood-prone city runs out of water: Inside Chennai's 2015 flood" – contextual info on urban drainage challenges (used for background).
- **IIT Delhi Aab-Prahari Project:** Prototype citizen-flood-reporting system (shows feasibility of crowdsourced flood data).
- **General Transit Feed Specification (GTFS):** Standard for integrating public transit schedules.
- **Firebase PWA Documentation:** for offline/web app guidance (tool for caching, authentication etc.).
- **OpenStreetMap / Leaflet Docs:** for interactive mapping and routing.
- **NDMA Guidelines (2021):** National Disaster Management guidelines on floods, emphasising the role of community reporting in early warning.



Each figure and data point above is drawn from or consistent with these sources. For example, Mumbai flooding details come directly from Times of India, and Bengaluru effects from Hindustan Times. VARSHA's design aligns with recognized needs by NDMA and municipal flood commissions: leveraging citizen reports and improving forecast dissemination.